基于 MongoDB 的问卷系统
第二阶段完成报告

郭婷月 10244507404

仓库链接：https://github.com/Caraice/GTY_Questionnaire_System_MongoDB

项目概述

这一阶段，我根据题目复用、共享和版本管理的需求，继续修改第一阶段的问卷系统。原来的注册登录、创建编辑、规则跳转、发布填写、答卷保存和统计仍然保留，新增题库、按版本导入、历史恢复、共享撤销、引用追踪和跨问卷统计。当前提交版本为 0.11.0，仍使用 FastAPI、PyMongo 和原生网页，业务数据统一存入 MongoDB。

这次的重点是让同一道题能够反复使用，同时保证旧问卷和历史答案不跟着新版变化。因此，我保留问卷里的内容快照，在外部增加独立题目和不可变版本，再把归属、共享和来源分开处理。本报告按“原来怎样设计、遇到什么问题、为什么调整、怎样验证”的顺序说明修改过程，不重复完整 API 和逐条测试命令。

实际验证结论

检查项	已取得的结果
提交副本后端	409 passed，0 failed/errors/skipped；151.39 秒，真实 MongoDB
前端状态逻辑	25 passed，0 failed/skipped；130.7946 毫秒
新旧兼容贯穿	四题型 A/B 链通过；旧问卷、答卷、v1 BSON 与旧统计保持不变

一、第一阶段设计与需求变更

1.1 第一阶段的设计是什么

第一阶段，我主要围绕一份问卷组织数据：在 surveys 中嵌入题目、选项、校验限制和跳转规则，用题目数组表示顺序；再由 responses 独立保存每次提交，答案按问卷内的稳定题目 ID 定位。发布时，系统固定 published_version，之后修改期限或关闭只改变管理版本，不能修改发布定义。当时这样设计，是为了方便整卷读取和单文档更新，也让每次提交都有确定的题目内容可供解释。

1.2 新需求带来了哪些问题

梳理变更需求时，我发现题目已经不再只属于一份问卷。单纯复制题干虽然能复用文字，却无法判断是不是同一道题、用了哪个版本、从哪里改来；如果让多个问卷直接读取同一份可变题目，旧答案又可能失去原来的含义。共享、恢复历史和跨卷统计都需要明确题目的身份与版本，原有的问卷内题号已经不够用了。

变更方向	本阶段对应的处理
保存与重复使用	题目独立保存，问卷编辑可预览并选择具体版本导入
共享给其他用户	按账号授权指定版本；接收方可导入，原作者继续使用
修改不影响旧卷	内容版本只追加；发布问卷保留快照，另存新题独立建身份
历史与恢复来源	显示题干、选项、限制差异；恢复生成新版本并记录来源
不同版本同时存在	旧版与新版并存，不同问卷可选不同版本，不自动跟随更新
查看题目使用范围	按版本展示有权查看的问卷、状态和每个实例，重复使用单列
题库管理与更新	题目可加入多个题库，移出不删内容；草稿可主动升级
单题跨问卷统计	先逐版本汇总回答，再按类型、选项映射和单位检查是否合并

1.3 为什么需要修改原设计

因此，我把调整重点放在题目的独立身份和生命周期上，继续复用已有的填写系统。我保留适合整卷读取的嵌入快照，再增加用于复用和追溯的版本引用。这样既能留下旧答案对应的题目内容，也能说明新旧题目的关系。设计时，我没有按题干是否相同来合并题目，也没有让题库更新自动改动已经发布的问卷。

二、MongoDB 结构变化与兼容处理

2.1 数据结构如何变化

我保留了原来的 users、sessions、surveys、responses 和 app_settings，并在第二阶段增加以下七个集合。初始化后共有 12 个集合，schema_version 仍为 1；数据结构的代际、管理修订号和内容版本号各有用途，我没有把它们混用。

新增集合	保存内容及设计理由
题目身份	question_templates 保存所有者、当前头指针、edit_version 和待恢复预留
固定内容版本	question_versions 保存 number、内容摘要及父版本、派生和恢复来源
题库	question_banks 保存名称和所有者，不把所有题内容嵌入题库
题库成员	question_bank_memberships 保存多对多归属，移出只取消关联
版本共享	question_grants 保存版本接收方、active 和 revision，不授予答卷权限
历史引用	question_usages 保留迁移得到的旧问卷实例与来源版本关系
迁移记录	schema_migrations 记录显式迁移的完成情况，重复执行仍核对数据

为了区分“同一道题”“某个版本”和“在问卷里出现的一次”，我分别使用 template_id、template_version_id 和 instance_id。每次导入时，系统都会生成新的实例 ID，所以同题在同卷出现两次，排序、答案和跳转也能分别定位。我还把题库成员与内容分开保存，让归属调整不产生内容版本，版本更新也不改变归属。

这里，我继续采用“问卷嵌入快照、独立对象引用”的组合。题目和规则需要一起读取、一起冻结，嵌入便于单文档更新；题库归属、版本和共享各有生命周期，使用引用更便于管理，也减少了无意义复制。课程要求使用 MongoDB，它的文档结构也适合不同题型的嵌套限制。因此，我保留这些嵌套结构，并由业务层检查引用完整性。

2.2 兼容性问题与迁移办法

迁移时，我首先遇到的问题是：旧题只有问卷内 ID，无法可靠判断它最初来自哪道独立题。因此，002 迁移按旧问卷和实例建立确定性来源，只插入新增记录，不按标题合并，也不更改旧题号、选项顺序或答案类型。我让默认命令只做预检，显式 apply 才写入；重跑时核对并补齐缺失记录，不重复造数据。遇到来源变化或内容冲突，脚本会停止，保留原数据供检查。

在新导入流程中，我把 question_sources 与嵌入题目放进同一问卷文档，便于一起原子更新；读取时仍兼容旧 question_usages，用户明确升级后才以在线来源为准。旧发布卷缺 published_version 时，系统沿用原 version，管理操作不改变冻结定义。旧数字版本还可能没有 unit 字段，我没有批量回填，以免改变原摘要；跨版统计改为确认本次单位，同时保留合法 0 和旧内容的含义。

三、程序调整与版本安全

3.1 修改了哪些部分 程序如何调整

这次修改中，我继续沿用 API、Service、Repository 和 domain 分层：路由处理登录和输入，服务组织所有权、版本与状态，Repository 执行 MongoDB 写入和聚合，领域逻辑复用四题型校验与跳转。新增功能也按这一分工接入，包括题库、版本、共享、引用和跨卷统计。考虑到实际操作，我还要求页面提供“我的题库”“从题库添加”“版本历史、共享与引用”和“查看题目统计”等按钮，让用户直接在页面完成操作。

3.2 保存新版本 另存新题与恢复

我把两种保存操作明确区分开：“保存为新版本”沿用同一题目身份，生成新的固定内容记录；“另存为新题”建立新身份并保留来源，不继承原题权限。两种操作都保留旧版。恢复旧版时，我也采用追加版本的方式：例如已有 v1、v2、v3，恢复 v1 会生成 v4，内容取自 v1，前一版本仍是 v3，并另外记录恢复来源。这样还能继续查看中间的修改过程。

3.3 共享 引用与主动升级

共享功能中，我把授权范围限定到指定版本和已注册账号，原作者仍保留自己的题目；接收方需要修改时，先另存到自己名下。我也单独区分了题目使用权和结果查看权，引用列表只展示当前账号有权访问的问卷。撤销共享后，系统停止新的导入、更新或另存，保留已经导入的快照和历史答卷。预览与实际使用可能有时间差，因此使用时还要检查权限，准入与撤销以条件写入的先后为界。

对于草稿，我选择让用户主动决定是否升级。发现可用新版时，页面先提示，用户预览后再更新某个实例，系统保留其 ID 与题序，并重新检查选项和跳转引用；如果新题型或选项会破坏规则，就拒绝更新。已经发布或关闭的问卷保持原版本。这样，题库可以继续修改，旧卷也能保留原来的内容。

3.4 两个窗口修改与单机故障恢复

处理两个窗口同时编辑的问题时，我把内容编号 number 与编辑令牌 edit_version 分开。expected_version 只比较管理令牌；两个窗口持同一令牌保存，条件更新 CAS 只允许一个请求预留成功，另一个返回 409，版本编号另有唯一索引。考虑到现有单机 MongoDB 没有跨集合事务，服务先预留完整 pending_version，再按固定 ID 写入不可变版本，最后推进当前指针。中断后重新加载时，系统补齐同一预留，不重新编号或覆盖历史。

我还补充了写入结果不明确时的处理：指针已写入但回执丢失，旧令牌重试返回 409；内容碰撞时停止恢复，连接错误 503 不能直接当作未落库。响应只返回本次提交的令牌，避免把较晚写入者的新令牌配给旧内容。原有跳转与严格校验仍按实例执行，由服务端重建真实路径；0 正常参与比较和统计，跳过的必答题不被强行补答。

四、跨问卷统计与查询改进

4.1 先确认统计的范围与口径

设计统计入口时，我先把汇总范围限定到具体版本，再允许按需合并。MongoDB 从当前账号有结果权限的问卷出发，按冻结版本关联答卷，依据实际路径和实例 ID 取答案。我没有把题目共享当成答卷授权，排除列表也不会泄露其他发布者的问卷标题和数量。

指标	采用的口径
涉及问卷数	有合规来源实例的已发布或关闭问卷，包括还没有答卷的问卷
答卷数	参与问卷的冻结版本下的答卷记录数，含跳题，每份在本组只计一次
题目回答次数	真实路径中符合原版本限制的实例回答；重复题分别计，跳题和空白排除

梳理统计口径时，我特别区分了重复实例和重复答卷：同题重复导入会增加回答次数，不能把答卷总数一起放大。匿名回答也不做跨卷人员去重。已有手算样本中，v1 的 0、2、4、6 平均为 3，合并 v2 的 15 后是 3 卷、3 答卷、5 次回答，平均 5.4；0 进入分母，跳题不补 0。选择题给出各选项次数，文本和数字支持分页查看原值，数字另给均值。

4.2 什么时候允许不同版本合并

合并不同版本前，我要求系统先确认题型一致。选择题按稳定选项 ID 或显式完整一一映射对应，不能只看文字相同；映射缺项、多对一或有额外 ID 时都不纳入。数字题的单位必须明确且一致，kg 与 g 不能直接合并；旧版缺单位时，只确认本次统计单位，不改旧记录或自动换算。每份答案仍按自己的原版本校验，避免用旧版上限 10 误排新版合法的 15；页面同时说明参与版本和排除原因。

4.3 分页 索引和缓存的判断

随着列表和内容增多，我把题库、题目、引用和跨卷内容接入分页，历史按不可变 number 游标翻页。索引调整对应实际查询：列表使用所有者加时间，引用使用所有者加来源版本，聚合继续使用既有答卷复合索引。固定数据为 3000 题、5000 版、2000 问卷、10000 答卷，七次取中位；结果显示题目深页 4.302→1.131 毫秒，引用 2.503→1.661 毫秒，引用扫描由 1000 降为 100，历史游标只读 20 条。

不过，跨卷聚合从 46.160 变为 46.762 毫秒，没有显示新增索引带来加速。我据此保留了暂不增加额外缓存的方案，避免再引入提交、撤权后的失效问题。这组结果只是本机 Repository 查询，不含完整 HTTP，还受并行测试和热缓存影响，不能当作压测或容量承诺。原整卷接口仍整批返回内容，新增分页的范围也需要说清楚。

五、实验结果与问题修正

5.1 双账号贯穿与实际页面核对

为了把修改串起来验证，我安排了 A 保存共享、B 导入发布的完整流程。AI 协助执行的贯穿测试分别使用单选 a、多选[a]、文本“旧回答”和数字 0：A 保存并共享 v1，B 导入发布并提交；A 保存 v2 后，B 旧统计 JSON 完全相等。B 未获 v2 共享时升级返回 404，授权后草稿升级保留实例 ID，旧发布卷升级返回 409。A 恢复 v1 生成新 v3，撤销两版共享后，B 新导入均返回 404，空草稿没有新增题。旧问卷、旧答卷和 v1 原始 BSON 逐字节不变，数字旧平均值仍是 0。

我也把页面操作列入验证范围。已有页面检查使用 21 题库、25 版本、同版本 22 实例和 101 份文本答卷核对翻页：文本第二页仅剩第 101 条，总答卷和回答次数仍是 101。共享、恢复、跨版合并等真实页面证据继续按原轮次保留。

5.2 如何保证旧功能仍然可用

为了确认旧功能没有受影响，我要求回归同时覆盖第一阶段的创建问卷、添加题目、跳转、严格校验、提交保存和整卷/单题统计六类用例，以及注册权限、发布关闭、期限、匿名、多次填写、幂等重试和本人历史。提交副本的实际结果为 409 项 Python、25 项 Node 通过，无失败、错误或跳过，保留 1 条既有 Starlette/httpx 弃用警告。测试使用隔离库，没有清空正式库。

我在修改前保留了源码基线，第一阶段原文件逐项比较仍保持不变，0.8、0.9、0.10 基线和摘要也继续保留。兼容性检查进一步比较重复迁移、缺省字段读取和 BSON 内容，不能只凭接口返回 200 就判断兼容。上述 409 已包含 4 项贯穿和 13 项分页恢复用例，统计结果时没有再重复相加。

5.3 遇到问题后怎样修正判断

版本保存让我更注意“报错”和“实际写入”之间的区别：内容插入成功，当前指针仍可能没有更新；网络报错，也可能已经写入。因此，我要求故障测试覆盖预留、版本插入和指针提交前后，恢复始终使用同一预留 ID。后续检查还发现，响应期间可能出现较晚写入，于是让本次旧内容只返回自己的令牌，下一次过期保存返回 409。判断是否恢复成功，需要同时核对历史和指针，不能只看页面提示。

引用和统计的检查也促使我补充了来源校验。只保留一个来源标记，仍可能把本地修改过的题目算进原版本，所以实现会比对不可变版本摘要与问卷实际内容，过滤本地改写，并在查询开始按所有者限制范围。对于文字相同、ID 不同的选项，我要求使用稳定 ID 或明确映射，让每份答案继续按原版本的含义参与统计。

页面验证中，分页接口虽然通过，翻页却会让内容折叠，因此又调整为保持展开。初轮回归也曾因隔离 Mongo 未启动出现连接错误，恢复原数据目录后重跑，失败日志仍保留。这些问题让我在提出验证要求时，更重视接口、页面和数据库记录之间的核对。性能结果同样如实解释：统计没有加速，就保留这个结论，暂不增加缓存。

六、AI 使用与阶段总结

这一阶段，我继续使用 Codex 协助开发。我负责在对话中明确沿用第一阶段、不清库、稳定 ID、expected_version、版本冻结、结果权限、单位映射、实际测试和真实记录等要求，并根据暴露的问题补充修改方向。具体代码生成、测试执行、页面自动操作和材料整理由 AI 协助完成。报告保留这种分工，完整提示与修正过程记录在 AI 使用日志中。

阶段	实际改进及判断
迁移与兼容	先保存基线，再建立独立身份与来源；不拿旧问卷时间冒充原创时间
题库与版本共享	归属与内容分开，恢复追加新版本，权限在使用准入时再次核对
跨问卷统计	先逐版本，分清三项计数；用 0、跳题、重复实例和显式映射手算验证
并发与分页	补预留/指针恢复和令牌隔离；依据 explain 和固定数据决定暂不加缓存

6.1 第一阶段设计保留了什么 改进了什么

回看这次修改，我保留了整卷嵌入、发布快照、稳定实例、规则引擎和答卷独立存储，因为这些设计仍然适合填写场景；再补上题目身份、不可变版本、归属和授权关系，以及跨卷统计的来源口径。这让我认识到，需求扩展后需要重新划清对象的职责：问卷保存一次发布所需的确定内容，独立题目负责复用和版本历史，两者各自保留适合的结构。

AI 协助过程中，我也根据实际暴露的问题不断补充要求：我需要更清楚地解释设计变化、材料迁移，页面状态和恢复边界也要继续核查。我的需求梳理与改进要求、AI 协助完成的实现和验证，分别按实际分工说明。

6.2 当前边界与材料入口

目前，八项需求变更已有对应实现和验证。我把跨卷的“全部”明确限定为有结果权限的问卷，不开放他人答卷。当前提供数字原值和均值，没有额外直方图、自动单位换算、跨卷人员去重或统计缓存。新电脑安装、大型压力和正式业务恢复尚未验证。

最后，我沿用第一阶段的四目录整理提交材料：01 放可运行代码、schema 和配置，02 放系统说明、数据库设计、API 和关键逻辑，03 放步骤、输入输出、结果及原始证据，04 放本报告、真实 AI 日志、基线和 SHA256 清单。